

DataShare

Application de partage de fichiers par lien temporaire

Soutenance — Noémie Henry · React · NestJS · PostgreSQL · Docker

Le problème et la solution

- Problème : partager un gros fichier de façon sûre et temporaire.
- L'email limite la taille ; un lien classique reste accessible indéfiniment.
- Solution : un lien de téléchargement unique, qui expire au bout de 1 à 7 jours.
- Protégeable par mot de passe ; le fichier est supprimé automatiquement à l'expiration.

Fonctionnalités (MVP)

- Inscription / connexion (compte email + mot de passe).
- Upload de fichier (jusqu'à 1 Go).
- Génération d'un lien de téléchargement unique et temporaire.
- Mot de passe optionnel sur le fichier partagé.
- Espace personnel : historique, filtres (actifs / expirés), suppression.

Démonstration (live)

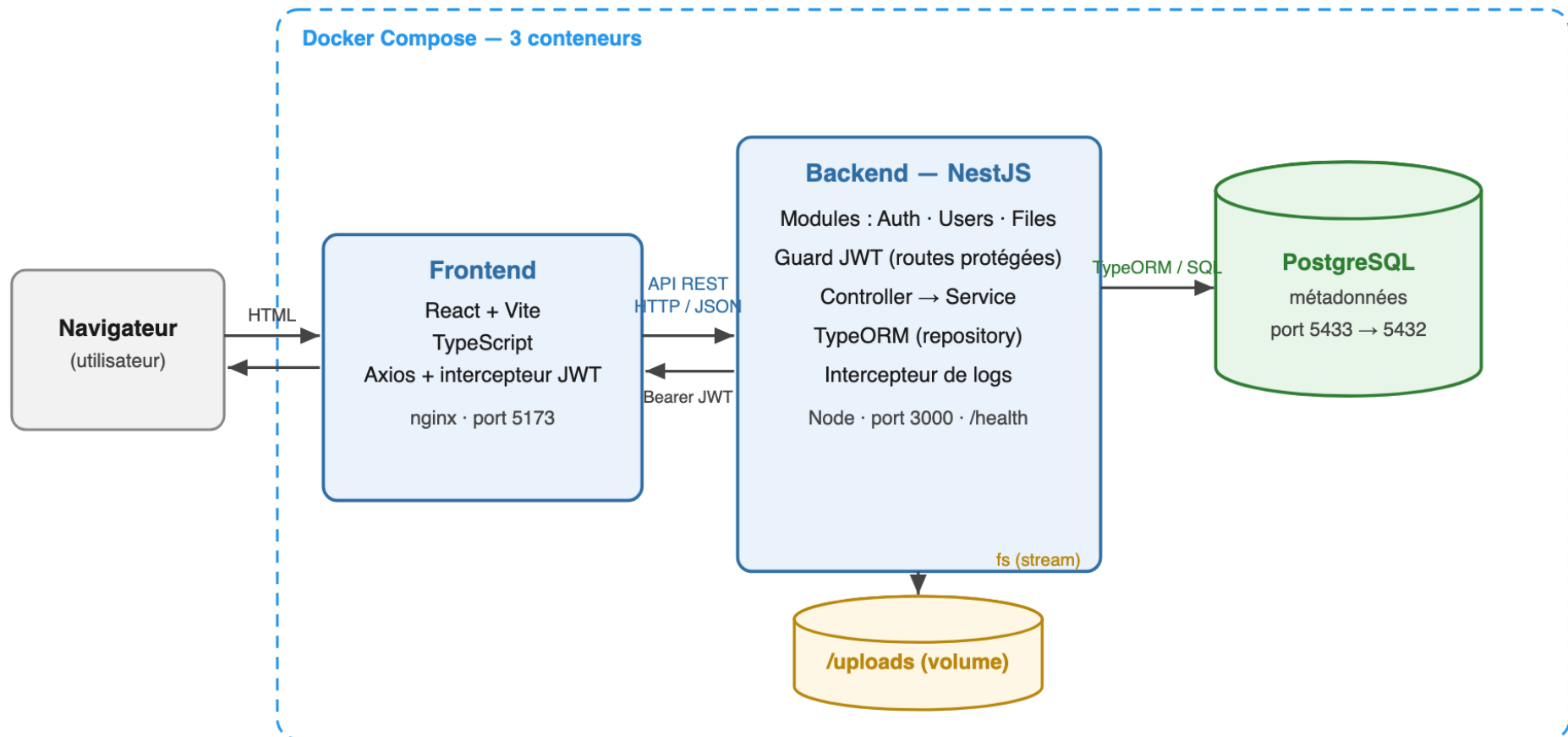
- Avant la soutenance : `docker-compose up -d` (ou `./setup.sh`) — appli sur `http://localhost:5173`.
- 1. S'inscrire / se connecter — le serveur renvoie un jeton JWT.
- 2. Déposer un fichier : mot de passe optionnel + durée (1 à 7 jours).
- 3. Copier le lien de téléchargement unique généré.
- 4. Ouvrir le lien (autre onglet) : saisie du mot de passe → téléchargement.
- 5. Mon espace : historique, filtres actifs / expirés, suppression.
- Filet de sécurité : garder une capture / vidéo de la démo au cas où.

Stack technique

- Frontend : React + Vite + TypeScript.
- Backend : NestJS + TypeScript (architecture modulaire).
- Base de données : PostgreSQL via TypeORM.
- Authentification : JWT + bcrypt.
- Conteneurisation : Docker Compose (3 services).
- Tests : Jest (unitaire), Supertest (intégration), Cypress (e2e), k6 (charge).

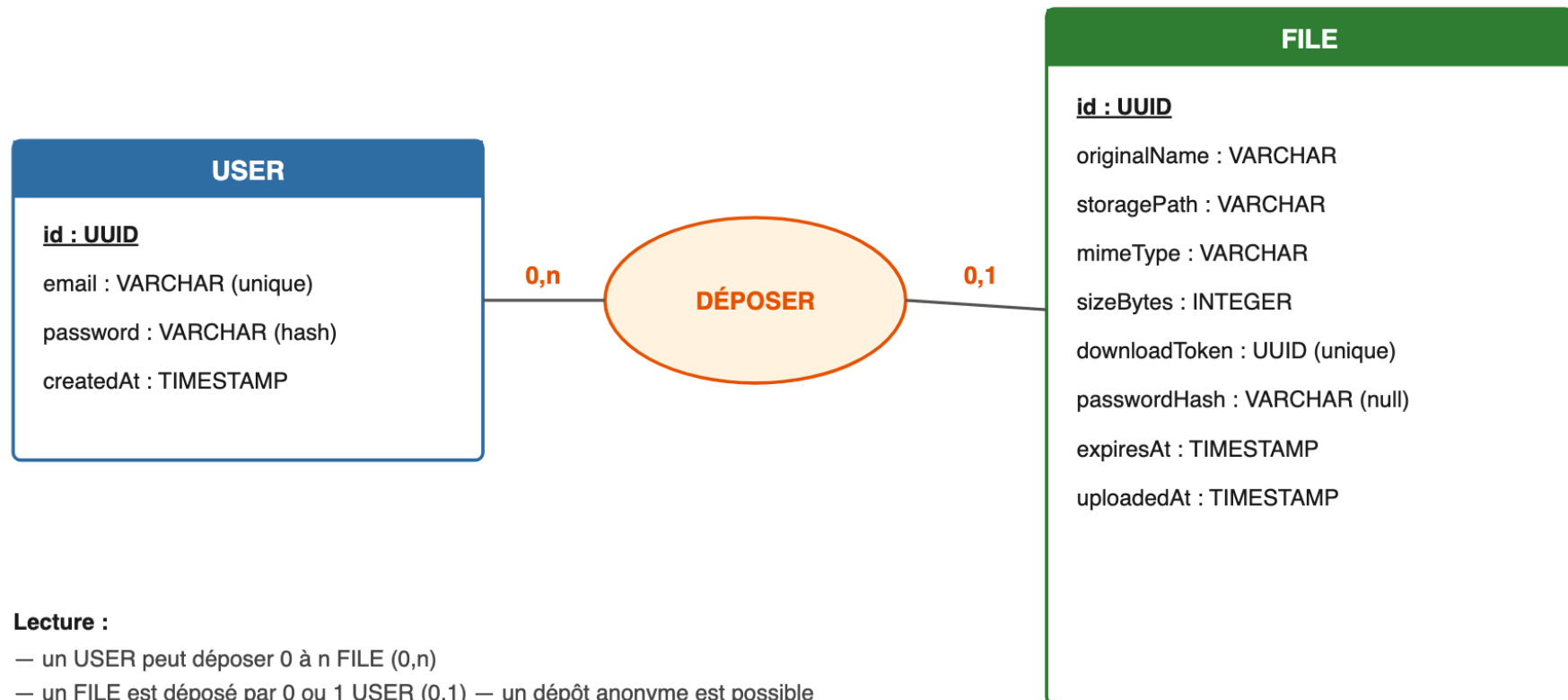
Architecture de la solution

DataShare — Architecture de la solution



Modèle de données (MCD)

DataShare — Modèle Conceptuel de Données (MCD)



Lecture :

- un USER peut déposer 0 à n FILE (0,n)
- un FILE est déposé par 0 ou 1 USER (0,1) — un dépôt anonyme est possible

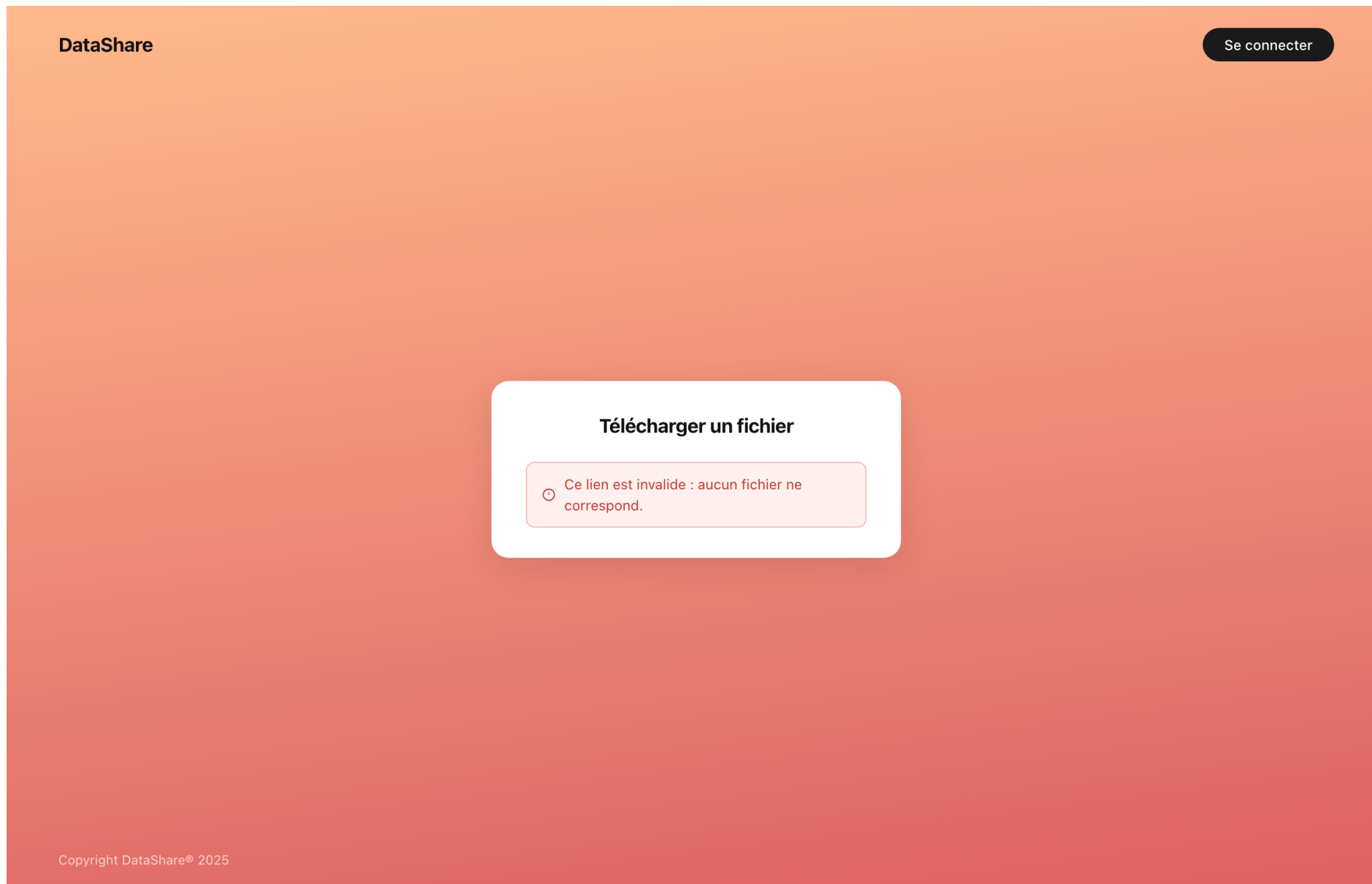
Traduction physique : la table files porte une clé étrangère user_id (NULL autorisé, ON DELETE CASCADE).

- Mots de passe hachés avec bcrypt — jamais stockés en clair.
- JWT signé ; secret lu depuis une variable d'environnement (.env).
- Liens = UUID v4 non devinables, avec expiration.
- Refus des exécutable par leur contenu réel (magic number), pas l'extension.
- Cloisonnement : un utilisateur n'agit que sur ses propres fichiers.
- Limite de taille (anti-déni de service) + journalisation structurée.

Gestion des erreurs et expérience utilisateur

- Lien invalide -> 404 « Ce lien est invalide » ; lien expiré -> 403 « Ce lien a expiré ».
- Mauvais mot de passe de fichier -> message clair (403).
- Identifiants invalides : même message pour email inconnu et mauvais mot de passe (anti-énumération).
- Validation des entrées côté client ET côté serveur.
- Jeton expiré -> déconnexion automatique et redirection vers la connexion (401).
- Feedback visuel : états de chargement, confirmation « Lien copié ! ».

Gestion des erreurs — exemple (lien invalide)



Tests et qualité

- 57 tests : 47 unitaires + 7 d'intégration + 3 end-to-end — tous au vert.
- Pyramide : beaucoup d'unitaires rapides, quelques e2e au sommet.
- Couverture de code ~99 % (objectif de 70 % largement dépassé).
- Tests de sécurité : exécutable refusé, taille plafonnée, expiration bornée, cloisonnement.
- Outils : Jest, Supertest, Cypress ; analyse statique avec ESLint.

Performance

- Test de charge k6 sur /auth/login (la route la plus coûteuse, à cause de bcrypt).
- 10 utilisateurs en parallèle pendant 30 s → 100 % de succès.
- Temps médian ~101 ms, p95 ~157 ms — tout sous le budget de 500 ms.
- Frontend : bundle JS gzippé < 100 Ko (Vite, tree-shaking).
- Téléchargement en streaming : le fichier n'est jamais chargé entièrement en mémoire.
- Logs structurés (JSON) : méthode, route, statut, durée — pour suivre la latence.

Choix techniques justifiés

- NestJS : structure modulaire imposée, TypeScript natif, tests et injection de dépendances.
- React + Vite : standard du marché, démarrage et build très rapides.
- PostgreSQL : relationnel, adapté à « un utilisateur possède plusieurs fichiers ».
- Magic number plutôt qu'extension : l'extension et le type MIME sont falsifiables.
- Fichiers sur disque, métadonnées en base : chacun fait ce qu'il fait le mieux.

Difficultés rencontrées et solutions

- Distinguer lien invalide / lien expiré → 404 vs 403 pour un message clair.
- Bloquer un fichier dangereux → détection par magic number (premiers octets).
- Secret JWT codé en dur → sorti dans .env via ConfigService.
- Deux fichiers de sécurité à 0 % de couverture → désormais testés (strategy, guard).
- Front et back non conteneurisés → docker-compose complet + endpoint /health.

DevOps et conteneurisation

- 3 conteneurs : PostgreSQL + backend (Node) + frontend (build React servi par nginx).
- docker-compose up : un environnement identique partout, déploiement reproductible.
- Health check /health : le frontend attend que le backend soit « sain ».
- Secrets en variables d'environnement, hors du dépôt Git.
- Prochaine étape : un pipeline CI/CD lançant les tests à chaque commit.

Limites assumées et perspectives

- Pagination sur la liste des fichiers (montée en charge).
- Stockage objet (S3) au lieu du disque local pour scaler horizontalement.
- Révocation de token (refresh token / liste de révocation).
- Accessibilité (ARIA / WCAG) et conformité RGPD (mentions légales).
- Migrations TypeORM au lieu de synchronize en production.

Conclusion et usage de l'IA

- Un MVP complet : fonctionnel, testé, sécurisé, documenté et conteneurisé.
- IA utilisée comme assistant : je définis la tâche, je relis, je teste, je valide.
- Les choix d'architecture et de sécurité sont les miens ; usage documenté dans AI_USAGE.md.
- Merci de votre attention — je suis prête à répondre à vos questions.